Task 12: Morphological Erosion

Shrinks foreground region boundaries within a binary image utilizing a square structuring element kernel.

এই প্রোগ্রামটির নাম Image Erosion using Morphological Operations (Python)। এটি Digital Image Processing-এর একটি গুরুত্বপূর্ণ Morphological Image Processing Technique।

এই প্রোগ্রামের মূল কাজ হলো Binary Image-এর White Object বা Foreground-এর আকার ছোট করা।

তোমার দেওয়া ছবিতে দেখা যাচ্ছে, Original Image-এর সাদা বর্গাকার অংশটি Erosion করার পর ছোট হয়ে গেছে। এটাই Erosion-এর মূল কাজ।

1. Image Erosion কী?

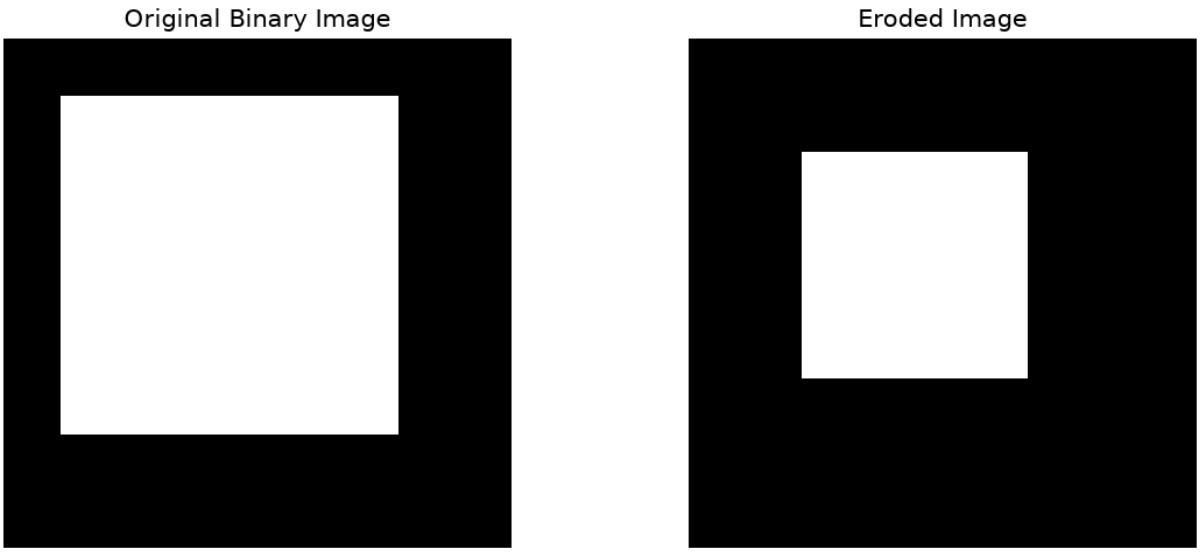
Erosion হলো একটি Morphological Operation, যার মাধ্যমে Binary Image-এর Foreground Object-এর আকার ছোট করা হয়।

সহজভাবে:

- White Pixel (1) = Foreground বা Object।
- Black Pixel (0) = Background।
- Erosion = White Object-এর Boundary ছোট করা।

ধরো, একটি White Square-এর চারপাশে Black Background রয়েছে। এখন আমরা যদি 3×3 Kernel ব্যবহার করে Erosion করি, তাহলে White Square-এর চারপাশের কিছু Pixel Black হয়ে যাবে।

Erosion-এর আগে ও পরে



মনে রাখবে: Erosion White Object ছোট করে এবং Black Background-এর পরিমাণ বাড়ায়।

2. Structuring Element বা Kernel কী?

Erosion করার জন্য আমরা একটি ছোট Matrix ব্যবহার করি। এটিকে Structuring Element বা Kernel বলা হয়।

তোমার প্রোগ্রামে 3×3 Kernel ব্যবহার করা হয়েছে।

kernel = np.ones((3, 3), dtype=np.uint8)

এর ফলে Kernel Matrix হবে:

3×3 Structuring Element

1	1	1
1	1	1
1	1	1

এখানে ৯টি Value-ই 1। অর্থাৎ সবগুলো Position-এ White Pixel থাকা প্রয়োজন।

এই Kernel Image-এর প্রতিটি Pixel-এর ওপর Slide করে কাজ করে।

3. সম্পূর্ণ Corrected Python Code

```
import numpy as np

import matplotlib.pyplot as plt
```

```
image = np.array([
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0]
], dtype=np.uint8)

kernel = np.ones((3, 3), dtype=np.uint8)
rows, cols = image.shape
k_rows, k_cols = kernel.shape
pad_y = k_rows // 2
pad_x = k_cols // 2

padded_image = np.zeros(
    (rows + 2 * pad_y, cols + 2 * pad_x),
    dtype=np.uint8
)
```

```
padded_image[
    pad_y:pad_y + rows,
    pad_x:pad_x + cols
] = image

eroded_image = np.zeros_like(image)

for i in range(rows):
    for j in range(cols):
        neighborhood = padded_image[
            i:i + k_rows,
            j:j + k_cols
        ]
        if np.all(neighborhood[kernel == 1] == 1):
            eroded_image[i, j] = 1
        else:
            eroded_image[i, j] = 0

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray', vmin=0, vmax=1)
plt.title("Original Binary Image")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(eroded_image, cmap='gray', vmin=0, vmax=1)
plt.title("Eroded Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

4. Code Explanation (Step by Step)

Step 1: Import Libraries

import numpy as npimport matplotlib.pyplot as plt

এখানে দুটি Library Import করা হয়েছে।

- numpy: Matrix তৈরি এবং Mathematical Operations করার জন্য।
- matplotlib.pyplot: Original এবং Eroded Image Display করার জন্য।

Step 2: Create Binary Image

image = np.array([[0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 1, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0]], dtype=np.uint8)

এখানে একটি 9×9 Binary Image তৈরি করা হয়েছে।

এই Image-এ দুটি Pixel Value রয়েছে:

- 0 = Black Pixel বা Background।
- 1 = White Pixel বা Foreground।

মাঝখানের White Pixel-গুলো একটি Square Object তৈরি করেছে।

dtype=np.uint8 ব্যবহার করে Pixel Value-গুলোকে 8-bit Unsigned Integer Format-এ রাখা হয়েছে।

Step 3: Create Kernel

kernel = np.ones((3, 3), dtype=np.uint8)

এখানে 3×3 Kernel তৈরি করা হয়েছে।

np.ones() ব্যবহার করার কারণে Kernel-এর প্রতিটি Value হবে 1।

এই Kernel ব্যবহার করে Image-এর প্রতিটি Pixel-এর আশেপাশের 9টি Pixel পরীক্ষা করা হবে।

Step 4: Get Image and Kernel Dimensions

rows, cols = image.shapek_rows, k_cols = kernel.shape

এখানে দুটি Image-এর Dimension বের করা হয়েছে।

প্রথম লাইনে Original Image-এর:

- rows = মোট Row সংখ্যা।
- cols = মোট Column সংখ্যা।

দ্বিতীয় লাইনে Kernel-এর:

- k_rows = Kernel-এর Row সংখ্যা।
- k_cols = Kernel-এর Column সংখ্যা।

আমাদের Image হলো 9×9 এবং Kernel হলো 3×3।

Step 5: Calculate Padding

pad_y = k_rows // 2pad_x = k_cols // 2

এখানে Padding Size Calculate করা হয়েছে।

যেহেতু Kernel Size 3×3:

$$pad_y = 3//2 = 1$$

$$pad_x = 3//2 = 1$$

অর্থাৎ চারপাশে এক Row এবং এক Column করে Padding দেওয়া হবে।

Step 6: Create Padded Image

padded_image = np.zeros((rows + 2 * pad_y, cols + 2 * pad_x), dtype=np.uint8)

এখানে Original Image-এর চারপাশে Padding দেওয়ার জন্য একটি নতুন Matrix তৈরি করা হয়েছে।

np.zeros() ব্যবহার করার কারণে নতুন Matrix-এর সব Pixel Value হবে 0।

আমাদের Original Image 9×9 এবং Padding 1 করে।

তাই নতুন Image-এর Size হবে:

$$(9 + 2) \times (9 + 2) = 11 \times 11$$

অর্থাৎ নতুন Matrix হবে 11×11।

Step 7: Insert Original Image into Padded Image

padded_image[pad_y:pad_y + rows, pad_x:pad_x + cols] = image

এখানে Original Image-কে নতুন Padded Image-এর মাঝখানে বসানো হয়েছে।

চারপাশে অতিরিক্ত যে জায়গা রয়েছে, সেখানে 0 থাকবে।

Padding কেন প্রয়োজন?

Image-এর Boundary বা Edge-এর Pixel Process করার সময় যেন সম্পূর্ণ 3×3 Neighborhood পাওয়া যায়, সেজন্য Padding ব্যবহার করা হয়েছে।

Step 8: Create Empty Output Image

eroded_image = np.zeros_like(image)

এখানে Original Image-এর মতো একই Size-এর একটি Blank Output Image তৈরি করা হয়েছে।

np.zeros_like() ব্যবহার করার কারণে সব Pixel-এর Initial Value হবে 0।

Erosion করার পর আমরা এই Image-এর মধ্যে Result Store করব।

Step 9: Visit Every Pixel

for i in range(rows): for j in range(cols):

এখানে Nested Loop ব্যবহার করা হয়েছে।

প্রথম Loop প্রতিটি Row Access করে।

দ্বিতীয় Loop প্রতিটি Column Access করে।

অর্থাৎ Image-এর প্রতিটি Pixel একে একে Process করা হবে।

Step 10: Extract Neighborhood

neighborhood = padded_image[i:i + k_rows, j:j + k_cols]

এখানে Current Pixel-এর আশেপাশের 3×3 Neighborhood সংগ্রহ করা হয়েছে।

অর্থাৎ প্রতিবার 9টি Pixel নিয়ে একটি ছোট Matrix তৈরি হবে।

এই Matrix-এর সঙ্গে Kernel-এর Position মিলিয়ে পরীক্ষা করা হবে।

Step 11: Apply Erosion Condition

if np.all(neighborhood[kernel == 1] == 1): eroded_image[i, j] = 1else: eroded_image[i, j] = 0

এটি আমাদের পুরো প্রোগ্রামের সবচেয়ে গুরুত্বপূর্ণ অংশ।

এখানে np.all() ব্যবহার করে পরীক্ষা করা হচ্ছে Neighborhood-এর সব প্রয়োজনীয় Pixel Value 1 কি না।

আমাদের Kernel-এর সব Value 1।

তাই 3×3 Neighborhood-এর 9টি Pixel-ই 1 হতে হবে।

যদি সবগুলো Pixel 1 হয়, তাহলে Output Image-এর Current Pixel হবে 1।

অন্যথায় Current Pixel হবে 0।

সহজভাবে:

- সব 9টি Pixel White হলে → Output White।
- একটি Pixel-ও Black হলে → Output Black।

এভাবেই White Object-এর Boundary ছোট হয়ে যায়।

Step 12: Display Original and Eroded Image

plt.figure(figsize=(10, 4))

এখানে Display Window-এর Size নির্ধারণ করা হয়েছে।

এরপর:

plt.subplot(1, 2, 1)plt.imshow(image, cmap='gray', vmin=0, vmax=1)

এখানে Original Binary Image Display করা হয়েছে।

plt.subplot(1, 2, 2)plt.imshow(eroded_image, cmap='gray', vmin=0, vmax=1)

এখানে Erosion করার পরের Image Display করা হয়েছে।

cmap='gray' ব্যবহার করে Grayscale Color Map নির্ধারণ করা হয়েছে।

vmin=0, vmax=1 ব্যবহার করে Pixel Value-এর Display Range নির্ধারণ করা হয়েছে।

সবশেষে:

plt.tight_layout()plt.show()

প্রথমটি Image-এর Layout সুন্দর করে এবং দ্বিতীয়টি Result Display করে।

5. তোমার ছবিতে ঠিক কী পরিবর্তন হয়েছে?

এটি পরীক্ষার জন্য খুব গুরুত্বপূর্ণ।

তোমার Original Matrix-এর White Object-এর Size হলো 6×6।

3×3 Kernel দিয়ে Erosion করার পর White Object-এর Size হয়েছে 4×4।

Erosion-এর Mathematical Result

Before Erosion

6 × 6

After Erosion

4 × 4

প্রতিটি পাশে 1 Pixel করে White Region কমেছে।

কারণ 3×3 Kernel-এর জন্য প্রতিটি পাশে 1 Pixel করে White Boundary বাদ যায়।

বাংলায় ব্যাখ্যা:

প্রথমে আমি NumPy এবং Matplotlib Library Import করেছি। এরপর একটি 9×9 Binary Image তৈরি করেছি, যেখানে 0 দ্বারা Black Pixel এবং 1 দ্বারা White Pixel বোঝানো হয়েছে।

তারপর একটি 3×3 Structuring Element বা Kernel তৈরি করেছি, যার প্রতিটি Value 1। এরপর Image এবং Kernel-এর Dimension বের করে Image-এর চারপাশে Zero Padding দিয়েছি।

পরবর্তীতে একটি Blank Output Image তৈরি করে Nested Loop ব্যবহার করে Original Image-এর প্রতিটি Pixel Process করেছি। প্রতিটি Pixel-এর জন্য 3×3 Neighborhood সংগ্রহ করেছি।

এরপর np.all() ব্যবহার করে পরীক্ষা করেছি Neighborhood-এর সব Pixel Value 1 কি না। যদি সবগুলো Value 1 হয়, তাহলে Output Pixel 1 করেছি। অন্যথায় 0 করেছি।

এভাবে প্রতিটি Pixel Process করার ফলে White Object-এর Boundary ছোট হয়েছে এবং Eroded Image তৈরি হয়েছে। সবশেষে Original এবং Eroded Image পাশাপাশি Display করেছি।

7. Important Viva Questions and Answers

Q1. What is Image Erosion?

Answer: Erosion is a morphological operation that reduces the size of foreground objects in a binary image.

বাংলা: Erosion হলো এমন একটি Morphological Operation, যা Binary Image-এর Foreground Object-এর আকার ছোট করে।

Q2. What is a Structuring Element?

Answer: A Structuring Element is a small matrix used to process an image during morphological operations.

বাংলা: Morphological Operation করার জন্য ব্যবহৃত ছোট Matrix-কে Structuring Element বলে।

Q3. Why do we use a 3×3 Kernel?

Answer: We use a 3×3 Kernel to examine the 9 neighboring pixels around each pixel.

Q4. What does 1 represent in a Binary Image?

Answer: 1 represents a White Pixel or Foreground Object.

Q5. What does 0 represent in a Binary Image?

Answer: 0 represents a Black Pixel or Background.

Q6. What is the purpose of np.all()?

Answer: It checks whether all selected pixel values satisfy a given condition.

Q7. Why do we use Padding?

Answer: Padding helps process boundary pixels by providing additional pixels around the image.

Q8. What happens when the Kernel Size increases?

Answer: The foreground object generally becomes smaller because more neighboring pixels must be white to preserve an output pixel.

Q9. What is the difference between Erosion and Dilation?

Answer:

- Erosion reduces the size of foreground objects.
- Dilation increases the size of foreground objects.

Q10. What is the output of this program?

Answer: The output is an eroded binary image in which the white foreground object becomes smaller.

8. Complete Working Process



Key Caveats:

1. Erosion সাধারণত White Foreground Object ছোট করে। তবে Binary Image-এর Foreground যদি Black হিসেবে নির্ধারণ করা হয়, তাহলে ব্যাখ্যা উল্টো হবে।
2. বড় Kernel ব্যবহার করলে White Object আরও ছোট হতে পারে।
3. Padding-এর জন্য এখানে Boundary-এর বাইরের Pixel-গুলোকে 0 ধরা হয়েছে।
4. Erosion ছোট White Objects বা সংযুক্ত অংশও সম্পূর্ণ মুছে ফেলতে পারে।

পরীক্ষার জন্য সবচেয়ে গুরুত্বপূর্ণ ৫টি Code Line:

kernel = np.ones((3, 3), dtype=np.uint8)padded_image = np.zeros((rows + 2 * pad_y, cols + 2 * pad_x), dtype=np.uint8)neighborhood = padded_image[i:i + k_rows, j:j + k_cols]if np.all(neighborhood[kernel == 1] == 1): eroded_image[i, j] = 1else: eroded_image[i, j] = 0

এই পাঁচটি অংশ বুঝতে পারলে Erosion-এর সম্পূর্ণ মূল Concept ব্যাখ্যা করতে পারবে।